

ChiP-seq 2026

Table of contents

1 Welcome to your first Bioinformatics Lab!	1
2 How to complete this notebook	1
2.1 Question 1	2
3 ChiP-seq workflow	2
4 Peak analysis	3
4.1 Question 2:	4
4.2 Question 3:	5
5 Analysis of TBP binding sites	5
5.1 Question 4:	5
5.2 Question 5:	5
5.3 Question 6:	5
5.4 Question 7:	7
6 Check your answers	8

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.ticker import FormatStrFormatter
```

```
try:
    import course_utils
except:
    !pip install "course-utils @ git+https://github.com/au-mbg/molecular-biology-II.git#
    subdirectory=course-utils"
```

1 Welcome to your first Bioinformatics Lab!

Right now you are in Mol-X lab cloning the DNA binding domain of the transcription factor TBP. This exercise shows what a researcher might do to understand the mechanism of gene activation by the transcription factor TBP. You will work with a real dataset from a ChIP-seq experiment investigating where TBP binds on human chromosome one.

2 How to complete this notebook

Throughout the exercise, you will answer 7 questions. The right answer will give you letters, numbers or symbols. **Note these down as you go and complete the exercise by entering the correct code at the end.**

2.1 Question 1

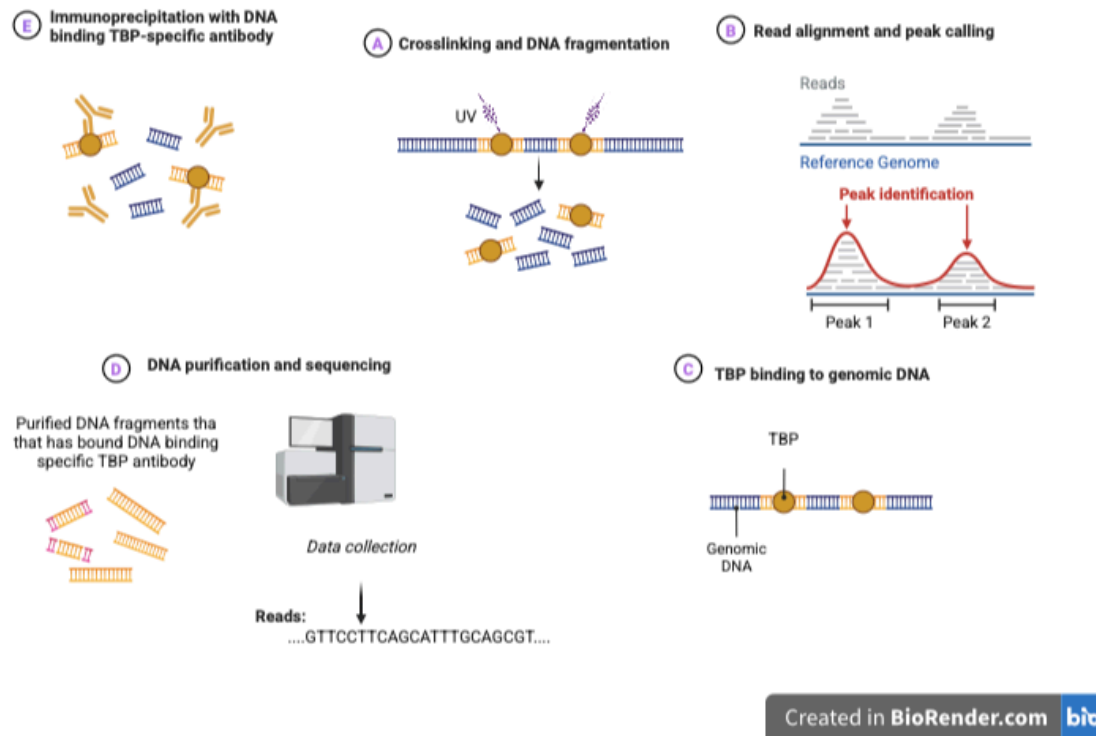


Figure 1: Disordered ChIP-seq workflow

Figure 1 shows the ChIP-seq workflow, unfortunately the figure has been scrambled such that the steps are out of sequence. You need to figure out the correct order of the shown steps.

Note down the purple letters in the order you created in the cell below by assigning them to `q1_answer`.

```
# Note the purple letters here
q1_answer = ""
```

3 ChiP-seq workflow

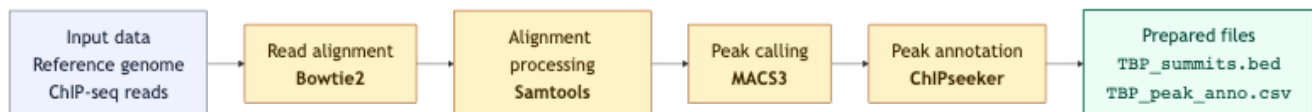


Figure 2: Diagram of ChiP-seq workflow starting from the reference genome and reads

Diagram Figure 2 shows the processing of ChIP-seq data. It uses several programs to process the data.

- Bowtie2: aligns the short DNA sequences obtained from sequencing, called reads, to a reference genome which in this case is human chromosome 1. The result describes where each read most likely originated in the genome.

- Samtools: processes the resulting sequence alignments. In this workflow, it is used to filter out uncertain alignments, sort the remaining reads by genomic position, and create an indexed BAM file that can be accessed efficiently.
- MACS3: identifies genomic regions in which unusually many aligned reads accumulate. These enriched regions are called peaks and represent candidate locations where TBP was bound to the DNA. MACS3 also identifies the summit of each peak, corresponding to the position with the strongest signal.
- ChIPseeker: compares the called peaks with known gene annotations. It classifies peaks according to the genomic features they are associated with, such as promoters, exons, introns, and intergenic regions.

The computationally demanding preprocessing has already been carried out. In this exercise, you will use Python to load the resulting files, visualise the locations of the peaks, summarise their genomic annotations, and interpret the results.

Diagram Figure 3 shows the part of the workflow that you will perform yourself.

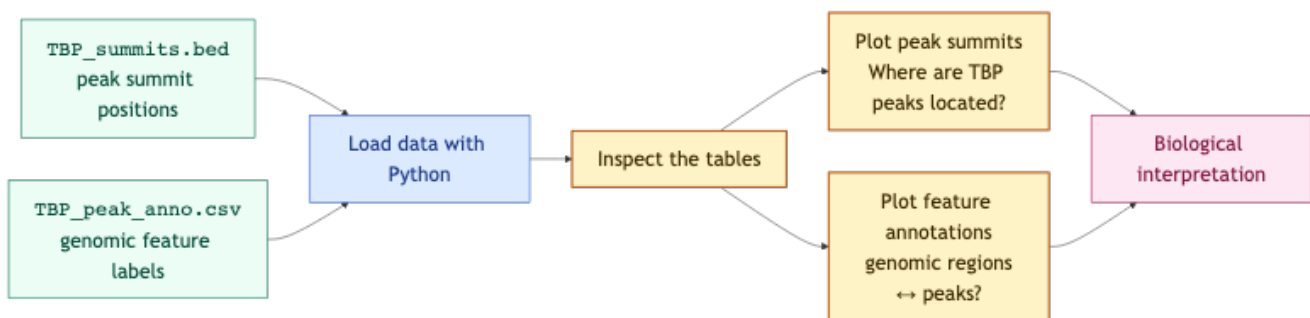


Figure 3: Diagram of analysis of processed ChIP-seq data

4 Peak analysis

The code above takes all the sequenced reads as input and aligns them to the defined reference genome, matching the sequence of each read to the corresponding sequence in the reference genome. The number of reads binding to specific regions of the genome is then quantified and visualized as peaks in a graph. Each cluster of peaks represents a potential binding site for TBP.

The cell below loads `TBP_summits_simplified.csv`, which is a simplified version of `TBP_summits.bed`, as a dataframe.

```

summits = pd.read_csv('https://raw.githubusercontent.com/au-mbg/molecular-biology-II/refs/heads/main/data/TBP_summits_simplified.csv')

display(summits)

```

The following cell plots summit position on the x-axis and peak score on the y-axis.

```

# Create figure and axis to make the plot.
fig, ax = plt.subplots(figsize=(10, 6))

# Plot the data - this creates a vertical line at each summit coordinate.
# the line starts at 0 and goes to "score".

```

```

ax.vlines(
    x=summits["summit_coordinate"],
    ymin=0,
    ymax=summits["score"],
    linewidth=0.8,
    color='black'
)

# Customization: Title & axis labels
ax.set_title("ChIP Peaks over region - TBP", loc="left")
ax.set_xlabel("Some chromosome")
ax.set_ylabel("")

# Customization: Axis limits & ticks
ax.set_xlim(0, 10_000_000)
ax.set_ylim(0, summits["score"].max() * 1.05)
ax.set_xticks([0, 2_500_000, 5_000_000, 7_500_000, 10_000_000])
ax.xaxis.set_major_formatter(FormatStrFormatter("%.1e"))

# Customization: Spines & layout
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
plt.tight_layout()
plt.show()

```

Exercise: Correct the label on the X-axis such that it says `Chromosome 1 position (bp)`.

Use the graph you've made to answer the following questions.

4.1 Question 2:

What does a high peak at a binding site typically indicate compared to a low peak? Note the symbol that corresponds to the correct option below.

%) A high peak indicates that fewer reads were aligned to the binding site, suggesting weaker binding affinity of the protein at that site.

!) A high peak suggests a strong enrichment or binding affinity at that site, meaning more sequencing reads aligned to this region.

?) A high peak represents a control region where the protein does not bind, while a low peak represents an area of high binding affinity.

“) There is no difference between high and low peaks; all peaks represent equal binding affinity.

```

# Note the answer symbol here
q2_answer = ""

```

4.2 Question 3:

How many regions have a peak intensity of more than 150?

```
# Note the answer symbol here  
q3_answer = ""
```

5 Analysis of TBP binding sites

Before analysing the TBP binding sites, answer the following questions.

5.1 Question 4:

Which region is found **closest to the start** of a gene and helps initiate transcription?

- A) Promoter ($\leq 1\text{kb}$)
- B) 3' UTR
- C) Other Exon

```
# Note the answer symbol here  
q4_answer = ""
```

5.2 Question 5:

Which region is typically found **far from any known genes** and may or may not be involved in gene expression?

- A) 1st Exon
- B) Promoter (2-3kb)
- C) Distal Intergenic

```
# Note the answer symbol here  
q5_answer = ""
```

5.3 Question 6:

Which region is located **after the coding sequence** of a gene and can impact mRNA stability?

- A) Promoter (1-2kb)
- B) 3' UTR
- C) 1st Intron

```
# Note the answer symbol here  
q6_answer = ""
```

Next we load the peak annotation data produced by ChIPseeker.

```

annotations = pd.read_csv("https://raw.githubusercontent.com/au-mbg/molecular-biology-II/
refs/heads/main/data/TBP_peak_annotations.csv")

annotations.head()

```

Then we define the features we are interested in and calculate percentages.

```

feature_order = [
    "Promoter (<=1kb)",
    "Promoter (1-2kb)",
    "Promoter (2-3kb)",
    "3' UTR",
    "1st Exon",
    "Other Exon",
    "1st Intron",
    "Other Intron",
    "Downstream (<=300)",
    "Distal Intergenic",
]

counts = annotations["feature_simplified"].value_counts()
percentages = counts / counts.sum() * 100
percentages = percentages.reindex(feature_order, fill_value=0)

```

Finally we make a plot of the features

```

# Pick colors for each feature
colors = {
    "Promoter (<=1kb)": "#a6cee3",
    "Promoter (1-2kb)": "#a6cee3",
    "Promoter (2-3kb)": "#a6cee3",
    "3' UTR": "#33a02c",
    "1st Exon": "#fb9a99",
    "Other Exon": "#e31a1c",
    "1st Intron": "#fdbf6f",
    "Other Intron": "#ff7f00",
    "Downstream (<=300)": "#cab2d6",
    "Distal Intergenic": "#6a3d9a",
}

# Create figure and axis
fig, ax = plt.subplots(figsize=(9, 7))

# Plot the percentage of each feature as a horizontal bar.
# The width of the bar is the percentage of that feature and
# it starts at the cumulative percentage of all previous features,
# stored in the variable left.

```

```

left = 0
for feature, percentage in percentages.items():
    ax.barh(
        y=["TBP"],
        width=percentage,
        left=left,
        color=colors[feature],
        label=feature,
        height=0.5,
    )
    left += percentage

# Customization: Title, limits and labels
ax.set_title("Feature Distribution", loc="left", fontsize=20)
ax.set_xlabel("Percentage(%)", fontsize=14)
ax.set_ylabel("")
ax.set_xlim(0, 105)
ax.set_xticks([0, 25, 50, 75, 100])

# Customization: Legend with feature names <-> colors.
ax.legend(
    title="Feature",
    bbox_to_anchor=(1.02, 0.5),
    loc="center left",
    frameon=False,
)

# Customization: Layout
ax.set_axisbelow(True)
plt.tight_layout()
plt.show()

```

Exercise: The plot you’ve produced colors all the promoter features the same color. Correct the feature colors so that “Promoter (≤ 1 kb)” stays the same color as before, “Promoter (1-2kb)” becomes a different shade of blue (e.g. #1f78b4), and “Promoter (2-3kb)” becomes light green (e.g. #b2df8a).

5.4 Question 7:

What kind of genomic element does TBP mostly bind to?

- A) Promoter (≤ 1 kb)
- B) Downstream (≤ 300)
- C) Other Exon

```

# Note the answer symbol here
q7_answer = ""

```

6 Check your answers

You have now answered all 7 questions. Run the next code cell and check to see if your answers are correct.

```
from course_utils import check_answer

answers = [q1_answer, q2_answer, q3_answer, q4_answer, q5_answer, q6_answer, q7_answer]
check_answer(answers)
```